



A coin selection strategy based on the greedy and genetic algorithm

Xuelin Wei¹ · Chang Wu¹ · Haoran Yu¹ · Siyan Liu¹ · Yihong Yuan²

Received: 19 January 2022 / Accepted: 27 May 2022 / Published online: 9 July 2022
© The Author(s) 2022

Abstract

Coin selection method refers to the process undergone when selecting a set of unspent transaction outputs (UTXOs) from a cryptocurrency wallet or account to use as inputs in each transaction. The most applied coin selection method that UTXO-based cryptocurrencies currently employ is an algorithm that decides on a certain set of UTXOs that matches the target amount and limits the transaction fee. However this approach trades off favourable maintenance overhead of the entire network for low transaction fees, as many low-value UTXOs known as “dust” is produced. Over time, this will impact the scalability and management of the cryptocurrency network as the global set of UTXOs become larger. Therefore, there is an urgency to find a higher-performing coin selection method suitable for UTXO-based cryptocurrencies. This paper proposes a method based on the greedy and genetic algorithm for effectively choosing sets of UTXOs in Bitcoin. The main objective of this coin selection strategy is to get as close as possible to the target while also maintaining and possibly reducing the number of UTXO inputs.

Keywords Coin selection · UTXO · Bitcoin transaction · Block size · Greedy algorithm · Genetic algorithm

Introduction

In 2008, Satoshi Nakamoto published the article “Bitcoin: A Peer-to-Peer Electronic Cash System”, which described a decentralized electronic currency architecture based on asymmetric encryption technology, P2P network, time stamp, hash algorithm mechanism, distributed consensus algorithm, and economic reward mechanism [1]. Meanwhile, this article also marked the official birth of Bitcoin.

In Bitcoin, a block is a storage unit used to publicly store several transactions permanently. Generating a block is considered as mining, which is a process of packing transactions into blocks and finding a random number that solves a cryptographic problem.

There may be multiple miners who are trying to generate a block concurrently. However, the probability of mining a block is proportional to the hashing power of the miner. Therefore, the miner with higher hashing power is more likely to find the nonce. When a miner packages a transaction into a block, the transaction is first confirmed then linked to the blockchain. When this block receives six consecutive block confirmations, the transaction is considered confirmed. A transaction generator can offer transaction fee to draw miners’ attention so that the transaction is more likely to be packed and confirmed.

Bitcoin is a decentralized application that uses the UTXO model, in which UTXOs are consumed as transaction input and created by transaction outputs. The payer can create and broadcast transactions through the P2P network without going through other third-party platforms. This kind of structure can replace the trusted third party. All transactions are stored in the block openly, transparently and distributed. Bitcoin addresses are equivalent to accounts in traditional financial systems, but addresses are not linked to the actual social identity, which protects the privacy of users.

At present [2], the total number of Bitcoin wallets is around 32 million. Since one user can have more than one wallet, the number of active users cannot be observed, it is also a rapidly growing number as more people become new users of Bitcoin. With the development of Bitcoin, large

✉ Chang Wu
changwu@uestc.edu.cn

Xuelin Wei
202021010312@std.uestc.edu.cn

Yihong Yuan
annieyuan4@gmail.com

¹ School of Information and Communication Engineering, University of Electronic Science and Technology of China, No. 2006, Xiyuan Ave, West Hi-Tech Zone, Chengdu, China

² School of Computer Science and Engineering, The University of Sydney, Sydney, Australia

transaction arrival rates have occasionally been observed. Bitcoin's block cannot accommodate all of them due to the limitation that the size of a block cannot exceed 1 MB [3]. The wallet in UTXO-based cryptocurrencies manages several UTXOs owned by the specific accounts. When the owner of a wallet wants to accomplish a transaction, the wallet will use their own coin selection method to choose a certain set of UTXOs to fund this transaction.

The coin selection method that Bitcoin uses is a random selection approach which is low in efficiency, as it cannot always ensure the target is met and with the increase of UTXOs the difficulty of finding an appropriate match also increases. The Bitcoin core defines “dust” as the trace amount of Bitcoin remaining in a wallet or address that is unable to process the transaction as its value is below the transaction fee required. We define dust as a UTXO whose value is lower than the transaction fee to spend it. For example, a Pay to Public Key Hash(P2PKH) input is 148bytes, an output is 34 bytes, and the additional information in transaction data is 10 bytes. A transaction with two outputs should pay at least 192 satoshi with default minimum transaction fee (1 satoshi per byte). In this example, if the P2PKH input is lower than 192 satoshi then it is considered “dust”.

Dust production is one of the shortcomings of the UTXO model. When a user needs to generate a transaction, the Bitcoin core will use its own strategy to select the set of UTXOs in the user's account [4]. It selects the UTXO that exceeds the target and generates two new coins one being paid and the other as “change”. Usually, it will assemble multiple coins to reach the target transaction amount. These complexities prove to have further ramifications when other factors are put into consideration such as transaction fees and production of dust. It needs to be noted that transaction fees are calculated based on the selection situation of coins in a transaction. Also, take notice that when large coins are spent rapidly there will be an accumulation of low value coins left that may become impossible to transact due to being lower than the required transaction fee.

These implications indicate the urgency for an efficient coin selection strategy for UTXO-based cryptocurrencies. A high-performance coin selection method would be one that puts into consideration selecting an appropriate amount of UTXOs to prevent high transaction fees as well as reducing the production of dust by spending low value coins earlier. This paper proposes a new coin selection strategy based on the greedy and genetic algorithm. It aims to make full use of low value coins to reach the transaction amount to prevent dust production and maintain or even possibly reduce the number of coins to avoid excessive transaction fees.

Size(Bytes)	Name	Description
4	version	Version ID
1+	tx_in_count	The number of transaction inputs
41+	tx_in	Transaction inputs
1+	tx_out_count	The number of transaction outputs
9+	tx_out	Transaction outputs
4	lock_time	Lock time

Fig. 1 Transaction structure of Bitcoin

Size(Bytes)	Name	Description
32	previous_output_hash	The hash value of reference transaction
4	previous_output_index	The index of reference output
1+	script_bytes	The length of script
variable	signature_script	Unlock script
4	sequence	sequence

Fig. 2 Structure of transaction input

Basic data structure in Bitcoin and the coin selection problem

Transaction structure of Bitcoin

UTXO model, which is shown in Fig. 1, is the transaction model invented by Satoshi Nakamoto. Each transaction has a version number, transaction inputs number, several transaction inputs and outputs and lock time. The version number and the lock time each occupy four bytes. The lock time defines the earliest time that a transaction is valid. A transaction with a lock time can be included in a block if the block height or the current time reaches the lock time requirement of the transaction. Otherwise, the miner will not include this transaction in the current block [5]. The purpose of this lock time is to prevent miners from fee sniping [6].

The number of bytes occupied by transaction inputs and transaction outputs can vary and depend on their respective amounts. The structure of transaction input is shown in Fig. 2, and the structure of transaction output is shown in Fig. 3.

Size(Bytes)	Name	Description
8	value	The value of transaction output
1+	pk_script_size	The length of locking script
variable	pk_script	locking script

Fig. 3 Structure of transaction output

The transaction outputs will generate new UTXOs, and the transaction inputs consume UTXOs, which are the source of coins. A single output cannot be used as an input for multiple transactions. If it does, the violation of this principle is called double spending [7].

Every Bitcoin transaction has a transaction hash which is calculated for the entire transaction content by Secure Hash Algorithm-256 (SHA-256). Transaction hash is also called transaction ID, and the size of the hash is 32 bytes. The UTXO as transaction input is indicated by the hash value of the referenced transaction and the index, which occupies 4 bytes of the UTXO in the transaction output. The sequence field is used to indicate the relative lock-time for this input, and it occupies 4 bytes. Transaction output has a value field, which indicates the value contained in the transaction output and occupies 8 bytes. The unlocking and locking scripts are used respectively to prove that the input UTXOs are owned by the payer and that the output UTXOs are locked to the beneficiary.

There is no relationship between the account and password in the traditional financial system, therefore the account and password are stored in the back-end database. Under these circumstances, different users cannot have the same account number. Since the Bitcoin system is decentralized, there is no third party to detect whether the account is duplicated, so a specific mathematical relationship between account and password is needed for identification and verification. A private key is a number, and the public key is generated by a one-way encryption function called the elliptic curve multiplication of the private key. For each transaction output, there is a locking script containing a hash of the public key of the beneficiary, declaring who is the beneficiary of this transaction. There is an unlocking script for each transaction input. The unlocking script provides the transaction's signature generated by private key to prove the beneficiary of the referenced transaction. Suppose someone tries to pretend he is a beneficiary and generates an unlocking script with his private key. In that case, the miner will find that the unlocking script does not match the locking script when the signature verification is performed and can discard the transaction [8].

There are two types of Bitcoin transactions. One is coinbase transactions, and the other is ordinary transfer trans-

Size(Bytes)	Name	Description
4	magic_number	Always 0xD9B4BEF9
4	block_size	Size of block
80	block_header	Block header
1+	transaction_cnt	The number of transactions
variable	transactions	Transaction details

Fig. 4 Block structure of Bitcoin

actions. The coinbase transaction is the first transaction of all related transactions in the block, created by the miner who generated the block. The transaction input of the coinbase transaction does not consume UTXO but contains an input called coinbase. The transaction output of a coinbase transaction is divided into two parts: the coin reward issued by the Bitcoin system for mining the block and the transaction fees of all the transactions in the block. Regular transactions consume UTXOs as transaction inputs (The transactions mentioned later refer to ordinary transfer transactions). A transaction may have multiple transaction inputs and multiple transaction outputs. The sum of these transaction inputs needs to be greater than or equal to the amount that the transaction needs.

Block structure of Bitcoin inputs

The structure of Bitcoin's block is shown in Fig. 4. As shown in Fig. 4, a block consists of a magic number, block size, block header and transactions, and the size of the first three fields is fixed. The size of a block has an upper limit. For example, Bitcoin requires that the size of a block be less than or equal to 1 MB, and the block header occupies 80 bytes. Since every transaction is recorded in the block, the transaction size affects the number of transactions that the block can accommodate [9–12]. The number of transactions is directly proportional to the size of the block [13].

After more than ten years of development, the transaction number has far exceeded the amount that can be accommodated in the space of 1MB. It means that some of the transactions will not be included in blocks immediately. In the Bitcoin system, transactions are prioritized as the following equation:

$$\text{Priority} = \frac{\sum_{i=1}^{\text{inputs}} V_i \times A_i}{S_f} \quad (1)$$

where V_i is the value of i th transaction input in this transaction, A_i is the age of i th transaction input in this transaction, and S_t is the size of this transaction.

Miners first pack the transactions with high priority into blocks, followed by those with high transaction fees. The transactions without fees and low priority are likely to wait a long time for confirmation [14]. Since Bitcoin's mining reward is halved every four years, transaction fees will become the primary means of income for miners in the future. [15]. Increasing transaction fees to attract the attention of miners is not a sustainable solution with possible future consequences such as overly high fees that render more transactions economically meaningless. The method proposed in this article takes the approach of increasing the number of transactions that a block can accommodate by reducing the transaction size. In turn, the transaction fee will also be lower than before with the size reduction of the transaction.

Dust and coin selection problem

The transaction fee is determined by both the size of this transaction and the fee-per-byte rate decided by the entire network. The higher the transaction fee a user set for a particular transaction, the sooner this transaction will be packed into a block by the miner. In this paper, we have defined dust as the UTXO with a lower value than the fee necessary to spend it. This transaction fee is calculated by assuming only one transaction input and one output in a fixed fee-per-byte rate is involved in the concerned transaction. Dust is usually produced by low-value transactions and the change output from them that cannot be pruned by the system automatically. It will cause the growth of the global UTXO set and leaves a part of inexhaustible UTXOs. If the coin selection strategies are properly designed, the system will be filled with such UTXOs, this may become a burden for normal users to manage [4].

In order to build a transaction in cryptocurrencies using UTXO model, the user or wallet must select a certain set of UTXOs from the account as the transaction input. In this selection process, the basic task is to reach the amount required for this transaction. Therefore, the coin selection problem is usually interpreted as a subset-sum problem. In practice, the wallet will design its own coin selection method and the transaction fee is the main consideration when designing the coin selection algorithm. But the amount requirement and transaction fee are not the sole factors contributing to the coin selection problem. Designing an appropriate coin selection algorithm should consider utilizing dust, ensuring user privacy, accounting for specific user requirements as well as the transaction fee. A well-designed coin selection algorithm can accomplish multiple optimization objectives while reducing the administrative cost of the

system. Hence, we consider the coin selection problem as a multi-objective optimization problem.

Coin selection strategy in Bitcoin

When creating a transaction in Bitcoin, first, the UTXOs belonging to this address should be found, this is to find the previous confirmed transaction outputs that point to the address. Then, the selection of UTXOs from those available follows a certain random method till the sum of the UTXOs allocated is greater than or equal to the target amount to be paid. The method used by Bitcoin to select UTXO as transaction input is described as follows, the target amount is the amount to be paid (without transaction fees) [16]:

- If there is a UTXO whose amount is precisely equal to the target amount, select the UTXO as transaction input.
- Find the UTXOs whose amount is less than the target amount, sum UTXOs' amount as $nTotalLower$. Then find the UTXO whose amount is greater than but closest to the target amount. Its amount is called $nLowestLarger$.
 - If $nTotalLower$ is less than the target amount and $nLowestLarger$ exists, select the UTXO whose amount is $nLowestLarger$ as transaction input; if $nTotalLower$ is less than the target amount and $nLowestLarger$ does not exist, it means the balance is insufficient to pay.
 - If $nTotalLower$ is greater than the target amount, then use the random approximation (up to 2000 times) to find UTXOs as transaction inputs. The sum of these UTXOs' amount been selected is closest to the target amount.

Overall, the selection method employed by Bitcoin prioritizes selecting UTXOs whose sum of the amount is closest to the target on a random selection recursion. This strategy cannot ensure accuracy of target value matching and lack effectiveness in minimizing transaction fee. Moreover, the algorithm used does not take into consideration change output of the transaction and therefore overlooks dust production. In [4], they quantitatively examine Bitcoin's coin selection method in regards to dust production and observed a trend of increasing number of unprofitable UTXOs.

Related work

Amidst the abundance of research on cryptocurrencies, there are few papers on coin selection methods specifically. Thus, our review of related work looks at selection strategies that prioritize target value accuracy [17,18], enhance the knapsack algorithm [19], and protect user privacy [20]. Our work

Algorithm 1 random approximation**Input:**

```

The set of available UTXO,  $U_s$ ;
The set of selected UTXOs,  $U_{included}$ ;
The target value,  $nTargetValue$ ;
1: for  $nRep = 1$ ;  $nRep \leq 1000$ ;  $nRep++$  do
2:    $nTotal = 0$ ;
3:    $fReachedTarget = false$ ;
4:   for  $nPass = 1$ ;  $nPass \leq 2$ ;  $nPass++$  do
5:     for  $i = 1$ ;  $i \leq size(U_s)$ ;  $i++$  do
6:       if  $nPass == 1$  then
7:          $bool = randombool()$ ;
8:       else
9:         if  $U_{included}$  has  $U_s[i]$  then
10:           $bool = false$ ;
11:        else
12:           $bool = true$ ;
13:        end if
14:      end if
15:      if  $bool == true$  then
16:         $nTotal += U_s[i].value$ 
17:        add  $U_s[i]$  to  $U_{included}$ 
18:        if  $nTotal \geq nTargetValue$  then
19:           $fReachedTarget = true$ ;
20:          if  $nTotal \leq nBest$  then
21:             $nBest = nTotal$ 
22:             $U_{best} = U_{included}$ 
23:          end if
24:          break;
25:        end if
26:      end if
27:    end for
28:  end for
29: end for

```

Output:

```

The set of UTXOs selected by algorithm,  $U_{best}$ ;
The sum of  $U_{best}$ ,  $nBest$ ;

```

proposes a coin selection method that aims to meet the exact target requirement whilst maintaining a small and stable number of UTXO inputs, and avoiding the production of dust. This paper interprets the method through an explanation of the algorithms concerned and illustrates the effectiveness through visualized data sets from simulations. The coin selection algorithm used in the Bitcoin core is roughly 2000 rounds of random recursions to find a set of UTXOs that comes closest to the payment target [17]. This selection strategy is discussed in detail in section IV.

Erhardt's thesis looks at a sample of different wallets and their implemented policies in regards to the effect on transaction fees and the UTXO pool [18]. It also introduces the Branch'n'Bound algorithm to find an exact match of the transaction target value. Drawing on quantitative experiment results, Erhardt makes suggestions to the Bitcoin Core's algorithm on their coin selection method which was later adopted by the developers as a secondary method. The proposed method employs the Branch'n'Bound algorithm to seek an exact match of the target value and falls back on the Random Draw if it fails to find an exact match. D.J.Diroff proposes an

enhancement of the knapsack algorithm to achieve the goal of minimizing cost by ensuring the change output of one transaction can fit the target value of a later transaction exactly [19]. This take on the classic knapsack problem takes into consideration minimizing change outputs that may be too low in value rendering it ineffective for another transaction. Taking a different approach, [4,20] looks at the current state of cryptocurrency networks evaluating existing concerns and the effectiveness of strategies to combat them. Abramova and Bohme's paper looks at coin selection strategies and the intertemporal problem while evaluating them from a privacy perspective.

The paper [4] discusses in detail dust in UTXO-based cryptocurrencies namely Bitcoin, Bitcoin Cash, and Litecoin. Their work demonstrates the importance of designing appropriate coin selection methods as the current state of such cryptocurrency networks faces an imminent threat of dust. While illustrating the difficulty of the network's global maintenance due to its complexity, the research paper highlights potential percussions of the existing shortcomings of the current coin selection methods. Our work answers the concerns brought to light, proposing a coin selection method that disincentivizes dust creation whilst enhancing the overall performance in the context of UTXO-based cryptocurrencies.

Coin selection strategy based on greedy algorithm and genetic algorithm

The coin selection complication is an optimization problem with three major objectives. Meeting the basic requirement of reaching the target value whilst ensuring the lowest possible difference, maintaining a relatively small number of dust in the wallet, and limiting the number of coins utilised in a transaction. In our context, a coin is represented by a UTXO input. Assuming a Bitcoin user U has n available $UTXOs$, the *target* denotes the value of the transaction to be made. For the coin selection process, the size of the search space should be no greater than 2^n to prevent the search space becoming significantly larger. Our work focuses on increasing the accuracy of matching the target value of each transaction, maintaining a small number of dust in a wallet, and disincentivising the production of dust.

There exist an abundance of techniques for multi-objective optimization, some recent ones include particle swarm optimization [21] and deep reinforcement learning [22]. However, compared to the classical greedy and genetic algorithms these recent optimization techniques respectively have disadvantages that render them a lesser match compared to the greedy and genetic algorithm. In the case of the particle swarm algorithm, it falls easily into local optimum when working in a higher dimensional space and has a low conver-

gence rate. On the other hand, deep reinforcement learning requires hefty computation, an extensive amount of time, and lack data to train the network. In this section, our proposed strategy to select coins based on the greedy and genetic algorithm will be introduced, as well as the characteristics of this new approach will be examined. Then, the specific steps of using these two algorithms for coin selection in a transaction will be described in detail, with explanation of use and meaning at each stage.

Algorithm concerned

In order to prevent the production of dust and utilise low value coins before they become impossible to spend, coins selected for a transaction should be as close to the target as possible whilst including lower value coins yet maintaining the number of UTXOs at a level that will not cause unnecessary transaction fees. The greedy algorithm can obtain the minimum number of transaction inputs. However, this algorithm will discriminate against low value UTXO's, and the sum of the UTXO combination amount is not always the closest solution to the target amount. Thus, the genetic algorithm finds a UTXO combination with the least number of UTXOs and the closest amount to the target amount.

(1) *Greedy algorithm*: Greedy algorithm is a fast and straightforward ranking algorithm to solve optimization problems. It generates solution by iteration, and the locally optimal solution under the current situation will be selected during each iteration [23]. When the target amount to be paid is determined, the UTXO with the largest amount from the payer's current UTXO pool will be first selected by using a greedy algorithm. Then a UTXO with the second-largest amount will be selected, and so on, until the sum of the selected UTXOs is greater than or equal to the target amount. The UTXOs being selected is the solution obtained by the greedy algorithm. Though it may not be the optimal solution, it can determine the minimum number of UTXOs as a solution.

(2) *Genetic algorithm*: Genetic algorithm (GA) is firstly introduced by Holland in 1975 [24]. GA is a type of evolutionary algorithm that is used to solve optimization searches. It mimics the process of population evolution, such as natural selection, crossover, and mutation. With the successive iterations of population evolution, the optimal solution will be found. The main idea is to encode the feasible solutions of the problem to fixed-length binary strings [25]. Each binary string represents an individual in the population, which is called a chromosome, and each integer in the chromosome is called a gene. According to the problem, the objective function and fitness function are necessary. The fitness is used to evaluate the chromosomes to determine the probability of being selected for the next generation. The three basic

genetic operations in GA are proportional selection, single-point crossover, and essential mutation.

- Proportional selection: Select some outstanding individuals from the current population and copy them to the next generation. The probability that an individual is selected is proportional to the fitness of the individual.
- Single-point crossover: Partition the individuals into pairs. Select successive pairs of individuals from the current population as parents, determine the crossing position and then cross the codes on the corresponding positions of the parents to generate new individuals.
- Basic mutation: For a gene that needs to be mutated, if the original gene value is 0, the mutation operation changes it to 1; otherwise, if the original gene value is 1, the mutation operation changes it to 0.

When using a genetic algorithm, some parameters need to be specified in advance, called decision variables, including population size M , crossover probability P_c , mutation probability P_m , and termination time T .

Specific steps to select UTXOs by greedy algorithm and genetic algorithm

This subsection describes the method to select UTXOs as transaction inputs when generating a transaction. Firstly, the greedy algorithm is used to determine the least number of UTXOs as transaction inputs. The number from the greedy algorithm is used to determine the objective function and fitness function of the genetic algorithm. Secondly, the genetic algorithm is used to search for the optimal solution. Because currency always has the smallest unit, it can be considered that the unit of the amount of each UTXO is the smallest unit of currency. As for Bitcoin, the smallest unit is satoshi where $1 \text{ BTC} = 10^8 \text{ satoshis}$. When using algorithms, only the amount of UTXO is needed to represent a UTXO, and the amount is non-negative numbers. In order to simplify the explanation, the rest of the paper uses mathematical symbols to indicate the amount of UTXO or the results related to it.

The algorithm's output is a combination of UTXOs with the least number and the closest sum to the target, this target represents the amount to be paid, and the transaction fees are not included. The flow chart of algorithm is described in Fig. 5. The problem-solving steps are as follows:

- S1: Set *result* to zero. Consider a User U with available N UTXOs wants to pay *target* to someone. Let x_i denote the value of the i th UTXOs. Let $UTXOS$ denote the list $\{x_i\}_{1 \leq i \leq N}$ sorted in ascending order and *size* denote the number of $UTXOS$ elements;
- S2: Traverse the $UTXOS$ and calculate the current balance of this user U as *total*;

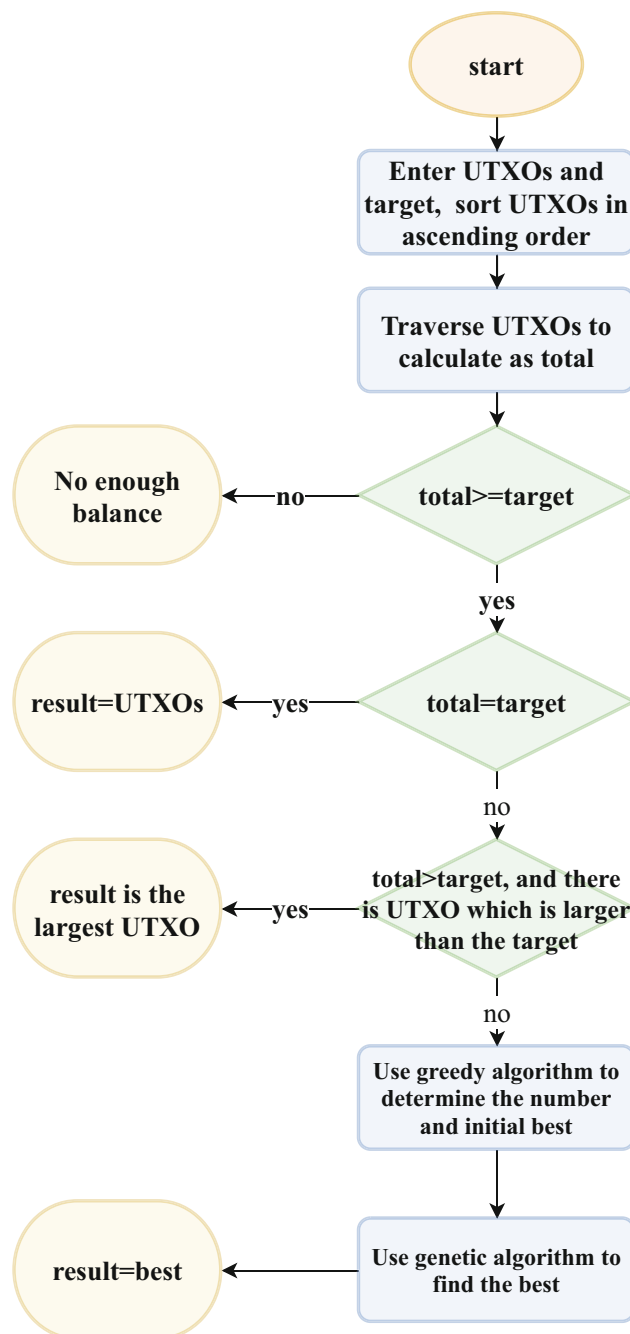


Fig. 5 Flow chart of whole process

- S3: If $total < target$, it means that the balance is insufficient, skip to step S8, otherwise proceed to step S4;
- S4: If $total = target$, $UTXOs$ is the transaction inputs, that is $result = UTXOs$, skip to step S8, otherwise proceed to step S5;
- S5: If $total > target$ and there exists a $UTXO$ in $UTXOs$ which is larger than the target, then use $UTXO$ which is larger than target but closest to target as selection result, skip to step S8, otherwise proceed to step S6;

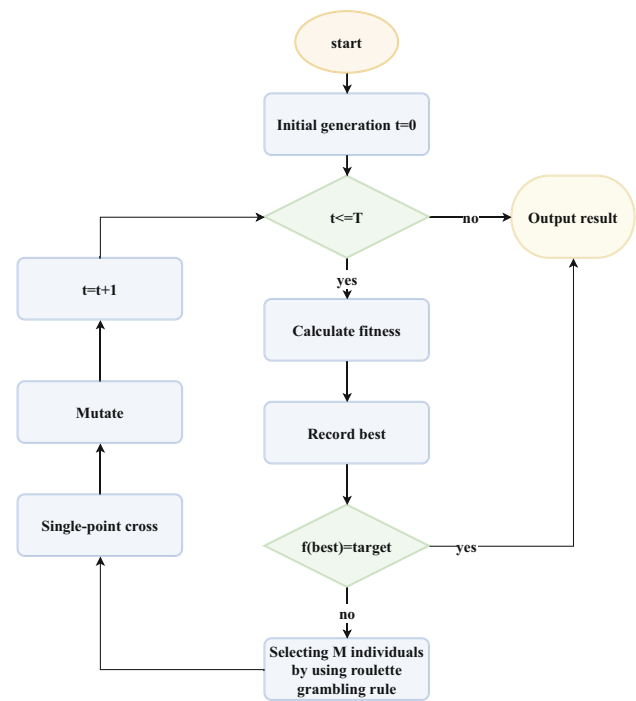


Fig. 6 Flow chart of genetic algorithm

UTXOs	Process	code
UTXO 0	unselected	0
UTXO 1	selected	1
UTXO 2	selected	1
UTXO 3	unselected	0
UTXO 4	selected	1
.....		
UTXO N-5	unselected	0
UTXO N-4	selected	1
UTXO N-3	unselected	0
UTXO N-2	selected	1
UTXO N-1	unselected	0

Fig. 7 Encoding process

- S6: Using the greedy algorithm, add the numbers in $UTXOs$ in order from large to small until the sum of the selected $UTXOs$ is greater than or equal to target, stop adding, record the number of selected $UTXOs$ as num , and the combination of positions of $UTXO$ in $UTXOs$ as one individual of the initial population and the initial value of $best$, $best$ means the optimal solution in the current population, and then randomly generate other $M - 1$ individuals to form the initial population;
- S7: Using the GA, starting from the initial population, the population is inherited, crossed, and mutated from

generation to generation, and the best individual in the previous generation is recorded as *best*, until the population generation equals the termination generation *T* or *best* is the best result, then *result* = *best*. The flow chart of GA is described in Fig. 6, and the specific steps of S7 are shown as follows:

- S70: Each individual in this algorithm represents a possible solution to this problem and is coded to a binary string of Length *size* as gene. If a *UTXO* in *UTXOS* is selected in this individual, the corresponding position will be coded as 1. If not, it will be coded as 0. The encoding process can be illustrated as shown in Fig. 7.
- S71: Since the initial value of *best* is obtained from S6, take the *best* as an individual in the initial population, and randomly generate other $M - 1$ individuals to form the initial generation, and set the population time to 0;
- S72: Determine whether the population time *t* is less than the termination time *T*. If it is less than *T*, continue to step S73. Otherwise, *result* = *best*, skip to S8.
- S73: Determine the objective function and fitness function, shown in function 2, and calculate each individual's fitness in the current population according to function 2. The fitness function:

$$f(I) = \frac{1}{target - \sum_{k=0}^{len} I_k \times x_k + \sum_{k=0}^{len} I_k}, \quad (2)$$

where *I* is an individual, I_k is the *k*th binary digit of *I*'s gene string, and the x_k is the value of *k*th *UTXO* in *UTXOS*. If *k*th *UTXO* is defined as a dust, the I_k in fitness function is considered as 0.

- S74: Update *best*. If $f(I) > f(best)$, then replace *best* with *I*.
- S75: If $\sum_{k=0}^{len} I_k \times x_k = target$, it is considered that the most suitable combination of *UTXOs* is found, *result* = *best*, skip to step S8, otherwise proceed to step S76;
- S76: Use the roulette gambling rule to select individuals to inherit to the next generation. This process is proportional selection. The total fitness of the current population is calculated as follows:

$$f_{total} = f(I_1) + f(I_2) + \dots + f(I_M), \quad (3)$$

where $I_1, I_2, \dots, I_M$ are the individuals in the current generation. And an individual fitness to f_{total} ratio is calculated as follows:

$$ratio_k = f(I_k) / f_{total} \quad (4)$$

where $1 \leq k \leq M$, *i* means the k_{th} individual in the current population. If using the roulette gambling rule, it is necessary to record the sum of the ratio, this is:

$$sum_{ratio}(k) = sum_{ratio}(k - 1) + ratio_k \quad (5)$$

where *i* has same meaning as function 4 and $sum_{ratio}(0) = 0$. The whole process of roulette gambling rule is making *M* selections to choose *M* individuals for next-generation, the specific process of a selection is shown as follows:

- * Generate a random number *rand* between [0, 1);
- * $k = 0$;
- * Calculate $sum_{ratio}(k)$, if $sum_{ratio}(k) \geq rand$, go to step 5, otherwise go to step 4;
- * $k = k + 1$, if $k = M$, go to step 5, else go to step 3;
- * Select the individual corresponding to *k* to the next generation population;
- S77: Single-point crossover. In the new *M* individuals obtained in step S76, every two individuals are used as parents, then randomly determine the crossing position and cross the genes in parents' corresponding positions. Firstly, we generate a random number *r* between [0, 1). Then if $r > P_c$, we randomly find a position between [0, *len*] and crossover these two genes to generate two new individuals. The crossover process is illustrated as shown in Fig. 8;
- S78: Basic mutation. Each individual obtained in step S77 generates a random number *r* between [0, 1). If $rand \leq P_m$, the individual is considered to have a mutation. Randomly select a mutation position, then reverse the binary digit at the mutation position. Then, update the P_m by $P_m = P_m \times (1 - \frac{1}{T+1-t})$. The mutation process is illustrated as shown in Figs. 9;
- S79: $t = t + 1$, skip to S72;
- S8: Output result as the transaction inputs.

There are some improvements to make GA perform better.

- (1) Encode a *UTXO* to its corresponding position in the gene as 0 or 1. It has two reasons.
 - The coding length will increase if a *UTXO* is encoded to its amount's binary representation and has a *UTXO* with a large value. Besides, if the distribution of *UTXOS* is sparse, the easier it is to generate unavailable individuals.
 - Because each *UTXO* can only be used once when generating an individual, each contains different parts, and the decimal representation of each part must be less than or equal to *size*. New individuals

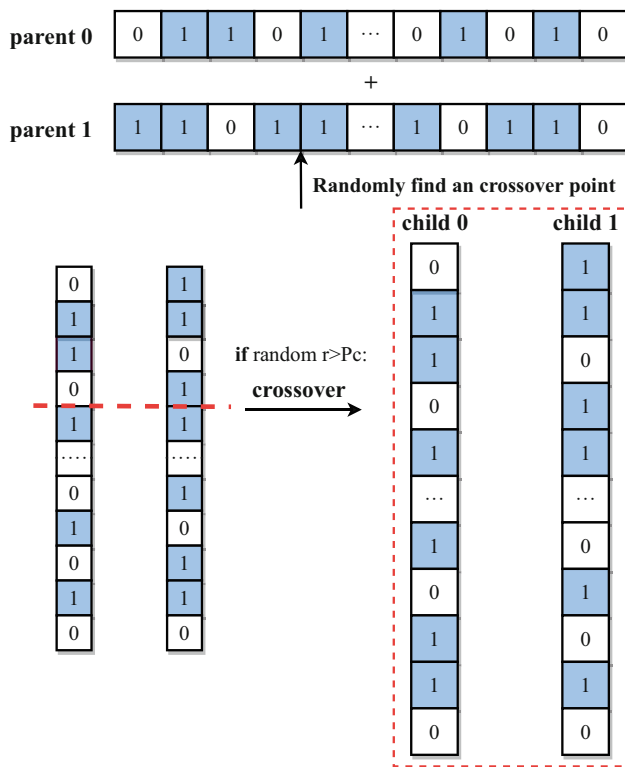


Fig. 8 Crossover process

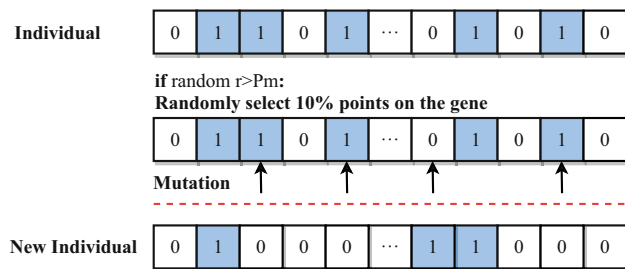


Fig. 9 Mutation process

must be checked whether there are duplicate parts and whether each part refers to an available position. If duplication occurs, the new individual is unavailable, regenerate a new one or remain its parents. However, every time a new individual is generated, it needs to check its availability, which will reduce the algorithm's efficiency. Therefore, when the *size* is small, the 0 can be added to *UTXOs* to make the *size* a power of 2. In this way, after obtaining a new individual, it is only necessary to check whether there are duplicates in the individual.

- (2) The result obtained by the greedy algorithm is used as an individual in the initial population and initial value of *best*. Because the genetic algorithm searches for individuals with high fitness in successive iterations, and the

result obtained by the greedy algorithm may be the best result.

The greedy algorithm will choose the *UTXOs* from an ascending list until the sum of *UTXOs* is larger than the *target*. In this case, the number of *UTXOs* selected by the greedy algorithm will always be minimum. The worst case is the greedy algorithm chooses all the *UTXOs* in the list. As for the genetic algorithm, the S74 step will traverse all individuals and compare it with *best*. If $f(I) > f(best)$, the *best* will be replaced by this individual. And in S75, the genetic algorithm will check if the current *best* is the suitable solution. if $\sum_{k=0}^{len} I_k \times x_k = target$, the final solution is found and then output the *best*. Even if the genetic algorithm can not find any individual that is better than the result given by the greedy algorithm, the final result will be the result with the minimum number of *UTXOs* that the greedy algorithm gives.

The Bitcoin *UTXO* selection method makes up to 2000 attempts(each attempt is called a 'round') to find a near optimal combination of input *UTXOs*. Although the time complexity of the Bitcoin method is $O(n)$, it can not guarantee that method can find solution that is close enough to the optimal solution. With *n* getting larger, the Bitcoin method will become more challenging to find a solution. Even if the solution is found, the distance from the *target* is uncertain.

The method we proposed combines a simple greedy algorithm and a genetic algorithm. The genetic algorithm is one kind of heuristic algorithm that is often used to solve problems with a significant large search space. The time complexity of the genetic algorithm is usually difficult to calculate, but we can still do some analysis on the time complexity of the method we proposed. Firstly, the time complexity of the greedy algorithm is the sum of the sorting process and finding process. We use the quicksort algorithm with time complexity $O(n \log n)$ for sorting. The finding process in our method is simply adding the value of *UTXO* and comparing it with *target*. The time complexity of the finding process should be $O(1)$, and the overall time complexity of the greedy algorithm is $O(n \log n)$. Let *g* denote the generation size. In each generation of genetic algorithm, three processes will be pursued: (1) Individual selection, (2) Crossover, (3) Mutation. We can discuss them separately.

- Individual selection: This process contains two steps: (1) Calculate the fitness of each individual, (2) choose the individuals. Because the number of individuals in each generation is the same, let *m* denote the number of individuals in one generation. The time complexity of the first step should be $O(mn)$, and the second step is $O(m)$. So the overall time complexity of this process is $O(mn)$;

Table 1 Results of bitcoin transaction selection methods (distance)

UTXO set	Round	Mean distance (BTC)	Mean distance (BnB)	Mean distance (greedy&GA)	Variance of distance (BTC)	Variance of distance (BnB)	Variance of distance (greedy&GA)
100numbers (1–1000)	100	9.65	368.74	0.01	321.28	6.56E4	0.01
1000numbers (1–1000)	100	22.13	315.78	0.00	1.92E3	5.54E4	0
100numbers (1–10,000)	100	84.24	3.60E3	2.15	2.05E4	5.86E6	7.06
1000numbers (1–10,000)	100	309.7	3.34E3	1.43	4.86E5	5.10E6	5.04
1000numbers (1–100,000)	100	2.11E3	3.80E4	10.86	6.85E6	4.97E8	121.50
1000numbers (1–100,000)	1,000	3.07E3	3.29E4	18.19	6.34E7	5.33E8	327.81
Real Account1	100	1.07E9	3.72E10	2.5394E7	6.01E18	1.11E21	8.94E14
Real Account2	100	2.35E9	1.42E9	7.28E4	7.60E18	3.57E18	3.41E9
Real Account3	100	1.29E10	9.00E9	2.28E7	1.75E20	7.34E19	7.98E14

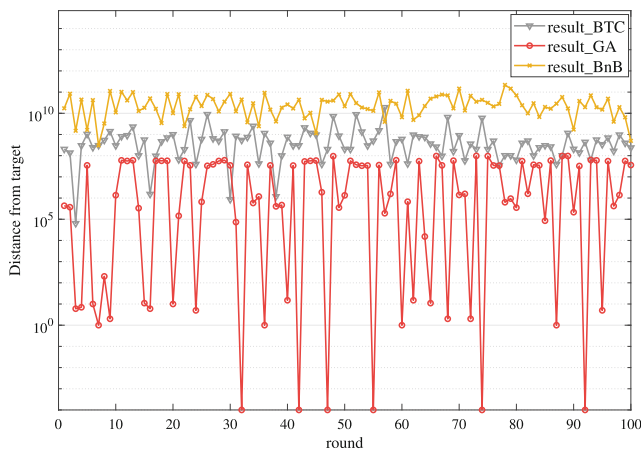
Table 2 Results from bitcoin transaction selection methods (UTXO quantity)

UTXO set	Round	Mean of UTXO quantity (BTC)	Mean of UTXO quantity (BnB)	Mean of UTXO quantity (greedy&GA)	Variance of UTXO quantity (BTC)	Variance of UTXO quantity (BnB)	Variance of UTXO quantity (greedy&GA)
100numbers (1–1000)	100	50.76	50.18	50.04	21.90	40.15	27.37
1000numbers (1–1000)	100	499.95	501.58	499.88	180.63	447.34	197.13
100numbers (1–10,000)	100	49.87	51.52	49.95	27.16	45.12	28.01
1000numbers (1–10,000)	100	499.49	499.7	500.08	215.67	329.51	214.78
1000numbers (1–100,000)	100	498.57	498.75	497.77	212.77	443.81	243.67
1000numbers (1–100,000)	1,000	500.35	499.32	500.22	264.21	407.99	254.93
Real Account1	100	125.21	124.41	75.48	74.29	344.97	511.95
Real Account2	100	152.59	123.53	125.36	1.52E3	339.95	49.55
Real Account3	100	101.59	87.24	73.44	333.6383	129.07	44.43

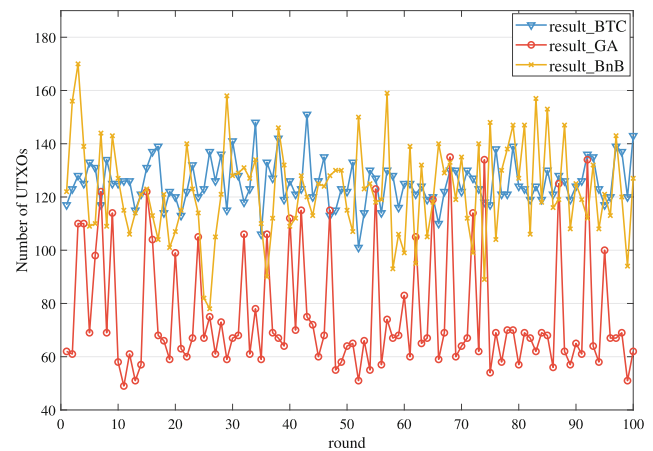
Real Account1 address: 1P5ZEDWTKTFGxQjZphgWPQUpe554WKDfHQ

Real Account2 address: 1NDyJtNTjmwk5xPNhjgAMu4HDHigtobu1s

Real Account3 address: 35pgGeez3ou6ofrjt8T7bvC9t6RrUK4p6



(a) Comparison of the distance from target



(b) Comparison of the UTXO quantities

Fig. 10 Comparison of 100 rounds results in real bitcoin account (bitcoin address: 1P5ZEDWTKTFGxQjZphgWPQUpe554WKDfHQ)

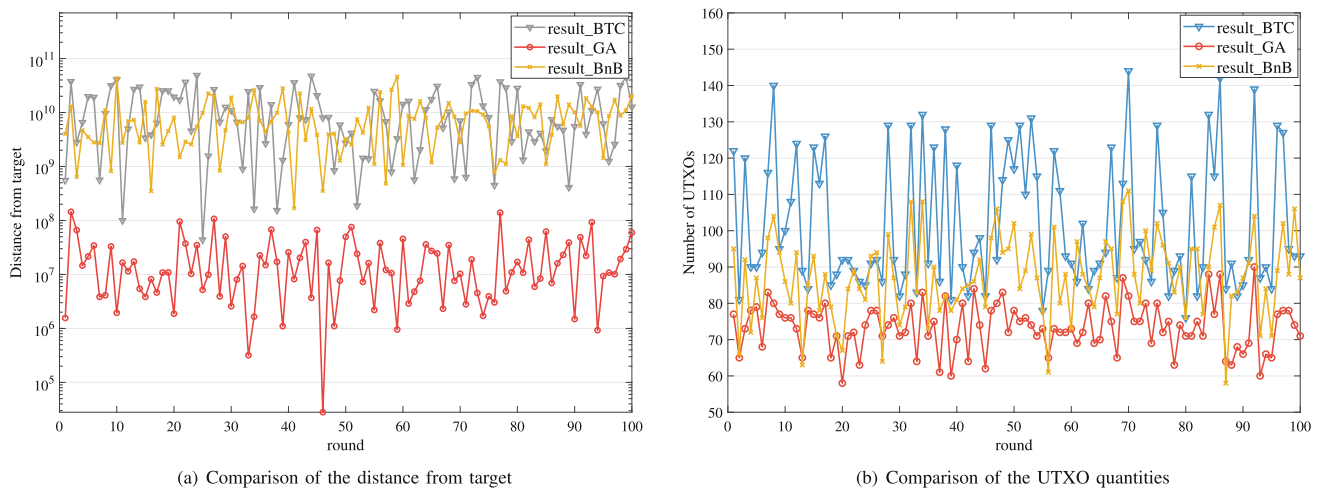


Fig. 11 Comparison of 100 rounds results in real bitcoin account (bitcoin address:35pgGeez3ou6ofrpjt8T7bvC9t6RrUK4p6)

- **Crossover:** The crossover process need to traverse the gene of all individuals in worst case. The time complexity of the crossover process is $O(mn)$.
- **Mutation:** In this process, we use a simple one-point crossover. In the worst case, we need to do a mutation process for all individuals. So the time complexity of this process is $O(m)$.

In summary, the time complexity of our method is $O(O(n\log n) + g \times (O(mn) + O(mn) + O(m))) = O(gmn)$.

Results analysis and discussion

During the first experiment, we conducted 8 sets of simulations, each of which was performed for 100 or 1000 rounds. For the first 6 sets of simulations, we randomly generated n positive integers between a certain range shown in the first column of Tables 1 and 2 as available *UTXOs* for each simulation. For the last three sets, we obtained three real Bitcoin accounts' *UTXOs* as available *UTXOs*. In each round, a random selection of *UTXOs* is summed up and set as the target amount. To examine the performance of our method against other strategies each experiment included simulations of Branch'n'Bound (*BnB*), the current algorithm Bitcoin uses (*BTC*), and our proposed method in order for vertical comparison (*Greedy&GA*, *GA*).

In total, we performed 1800 experimental rounds. The *UTXO* sets used in each set of simulations are shown in column 1 in both Tables 1 and 2, and the experimental rounds m in each set are shown in column 2. For each round, we randomly selected a set of *UTXOs* and set the *target* as the sum of their values. Ideally, the distance between *UTXOs* selected by the algorithm and the *target* is 0.

For the second experiment, we used the sample of *UTXO* set taken on October 1st of 2019 available on GitHub [19], we ran 1000 rounds of simulations. We created three identical wallets, with 2000 *UTXOs* and 10% of dust where the fee-per-byte rate is 22. In each round, a set of *UTXOs* is summed to assign as the *target* value of which our proposed algorithm and Bitcoin's method need to retrieve *UTXOs* to match. Then, if necessary the change output is added back to the wallets before three identical sets of 200 *UTXOs* including 10% dust is added to all wallets respectively. This is iterated for 1000 rounds, whereby the number of dust remaining in each wallet after each round is recorded.

In our method, the parameter of the genetic algorithm is set as the following based on experience:

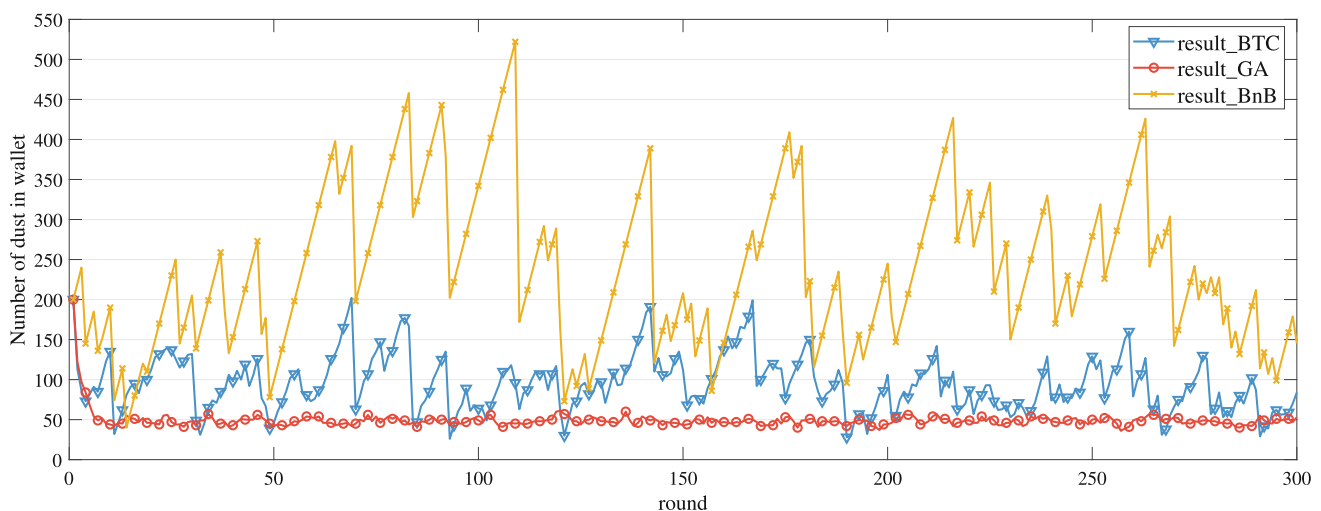
$$M = 200, P_c = 0.5, P_m = 0.01, T = 200 \quad (6)$$

We showed the mean distance between the *target* and the result of the Bitcoin method, Branch'n'Bound method and the method we proposed in columns 3, 4 and 5. Columns 6, 7 and 8 shows the variance from target of each method, it presents the stability of finding the closest solution to the *target*. Table 2 compares the three methods' mean and variance of *UTXO* quantity selected by the method. Figures 10 and 11 show the comparison of the results obtained by the real account 1 and real account 3. The *result_BTC* is the selection result obtained by the coin selection method used in Bitcoin, the *result_BnB* result obtained by the Branch'n'Bound method, and *result_GA* is the selection result obtained by our approach. Table 3 shows the minimum, maximum, and average of dust quantity in the three wallets in each 100 rounds. Figure 12 visualizes the dust quantity fluctuations in each wallet over 300 rounds.

According to the results from Tables 1, 2, Figs. 10 and 11, the method we proposed offers a solution whose result is

Table 3 Comparison of dust in wallet

Rounds	100	200	300	400	500	600	700	800	900	1000
Min (BTC)	26	26	26	26	26	26	26	26	26	26
Min (BnB)	40	40	40	35	35	35	29	29	29	29
Min (greedy&GA)	41	37	36	36	36	35	35	35	35	32
Max (BTC)	202	202	202	202	202	202	203	203	204	204
Max (BnB)	458	522	522	522	522	522	522	522	682	704
Max (greedy&GA)	200	200	200	200	200	200	200	200	200	200
Average (BTC)	99.54	100.21	94.50	91.87	90.96	91.99	93.87	93.77	97.98	99.14
Average (BnB)	237.86	234.26	236.75	225.35	226.15	219.60	217.82	212.33	225.30	229.17
Average (greedy&GA)	51.26	49.450	49.12	48.72	48.34	48.36	48.25	48.13	48.28	47.96

**Fig. 12** Comparison of 300 rounds result of dust

closer to the *target* and uses less *UTXOs*. In almost every simulation, the distance to the *target* of *UTXOs* chosen by our method is significantly closer than of the other methods whilst maintaining a similar mean number of *UTXOs* selected. In the first 6 sets of simulations, the greedy and genetic algorithms proved to be high-performing in accuracy of target-matching as the mean distance was low throughout. Additionally, the outstanding result of the methods variance of distance showcases its stability. This trend carries through into the simulations using real accounts. The two comparison of distance from target graphs in Figs. 10 and 11, visualizes the performance of all three methods and exhibits the distinguished performance of our proposed method in a vertical comparison.

From Table 2, it can be observed that our proposed method does not fall short in performance of *UTXO* quantity control. Hence, while hitting the target more accurately our method will not create transactions that require excessive transaction fees, this holds true stably. This performance can be clearly perceived in the graphs depicting the comparison

of *UTXO* quantities selected by all three methods in Figs. 10 and 11.

For each 100 rounds we compared the minimum, maximum, and average number of dust in each wallet to demonstrate the performance of the respective coin selection methods. Although Bitcoin's method has the lowest minimum number of dust however the average number of dust in the Bitcoin's method wallet is around 100. The result of Branch'n'Bound shows a the biggest range between minimum and maximum, it also has the highest average quantity of dust. Our proposed method stably maintains the number of dust in the wallet at a relatively low amount. Hence, demonstrating our coin selection method utilizes more dust when choosing the set of *UTXOs* used in each transaction without compromising low transaction fees. Figure 12 illustrates the result from the first 300 rounds. From the graph, it can be recognised that the results of methods *BTC* and *BnB* fluctuates majorly, while the *GA* method upholds a steady trend on a low amount of dust.

After the greedy algorithm is run, each *UTXO* selected has a large amount because small *UTXOs* were excluded,

which suppresses the utility of the low-value *UTXOs*. However, the genetic algorithm makes use of the lower value *UTXOs* regurgitating the suppressed small *UTXOs* from the greedy algorithm. Besides, because the initial value of *best* is set to the result obtained by the greedy algorithm, the final *result* must be the result of the greedy algorithm or a better result than the greedy algorithm. Although the method we proposed is better than the Bitcoin method in both distance and the number of *UTXOs*, this algorithm requires more time to solve the problem, as we discussed in section 4.

The integration of the greedy algorithm and genetic algorithm can formulate a solution that efficiently selects coins based on their individual values in a way to get as close as possible to the target with the least number of coins. Under this premise, the method also prevents the production of dust as it prioritizes using a combination of coins that comes to the specific target amount, therefore, eliminating excessive “change”.

Conclusion

The method described in this paper uses Bitcoin as the main subject and environment of experimentation. However, the proposed method is applicable to all blockchain applications that employs the *UTXO* transaction model and includes an amount field in the transaction. Although, transactions generated using this method cannot guarantee the lowest possible transaction fees, it does not produce excessively high transaction fees. The algorithm described in this paper is mature and has established accountability and hence why it is essential to illustrate the feasibility of this method. In genetic algorithm, due to the setting of the initial population and the initial value of *best*, the algorithm can guarantee the quality of the solution even when the termination time *T* is relatively small, and the improvement of the encoding method speeds up the process of solution searching.

As greedy algorithm and genetic algorithm are used, the time taken to search for the solution will be extended. If this time prove to be inconveniently long, then improvement on the selection, crossover, and mutation process is in need. Since P_c and P_m can affect the algorithm and its convergence, they can be changed automatically adaptively, that is, adaptive genetic algorithm [26]. In Bitcoin, the user's *UTXO* is stored in the *UTXO* pool. Selecting fewer *UTXO* as transaction input means that a larger number of *UTXO* will accumulate in the *UTXO* pool, which is not conducive to *UTXO* management or adds complexity to subsequent *UTXO* searches [27]. While this was not a complication in a controlled experimental environment, it could pose to be a hindrance in realising the benefits of this strategy in a real-world situation. Further research on improving the speed and

time of this coin selection method will be relevant for the near future.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Nakamoto S (2008) Bitcoin: a peer-to-peer electronic cash system. Decent Bus Rev 21260
2. Blockchain data [EB/OL]. <https://www.blockchain.com/zh-cn/charts>
3. Zima M (2018) (Short paper) inputs reduction for more space in bitcoin blocks. In: 2018 Crypto Valley Conference on Blockchain Technology (CVCBT). IEEE, pp 112–115
4. Pérez-Solà C, Delgado-Segura S, Navarro-Arribas G, Herrera-Joancomartí J (2019) Another coin bites the dust: an analysis of dust in *UTXO*-based cryptocurrencies. R Soc Open Sci 6:180817
5. Antonopoulos AM (2015) Mastering bitcoin: unlocking digital cryptocurrencies. O'Reilly Media Inc
6. Saad M, Njilla L, Kamhoua C, Mohaisen A (2019) Countering selfish mining in blockchains. In: 2019 international conference on computing, networking and communications (ICNC). IEEE, pp 360–364
7. Bamert T, Decker C, Elsen L, Wattenhofer R, Welten S (2013) Have a snack, pay with bitcoins. In: IEEE P2P, Proceedings. IEEE, pp 1–5
8. Wuille P, Lombrozo E, Lau J. Segregated witness (consensus layer) [EB/OL]. <https://github.com/bitcoin/bips/wiki/Comments:BIP-0141>
9. Özyılmaz KR, Patel H, Malik A (2018) Split-scale: scaling bitcoin by partitioning the *utxo* space. In: 2018 IEEE 9th international conference on software engineering and service science (ICSESS). IEEE, pp 41–45
10. Johannsson DV, Garzik J, Harding T. Dynamic maximum block size by miner vote [EB/OL]. <https://github.com/bitcoin/bips/wiki/Comments:BIP-0100>
11. Chang X, Zhao Y, School S, University F (2019) Scaling bitcoin: the state of development and future trend. Comput Appl Softw
12. Poon J, Dryja T (2016) The bitcoin lightning network: scalable off-chain instant payments
13. Hui Y, Zongyang Z, Jianwei L (2017) Research on scaling technology of bitcoin blockchain. J Comput Res Dev 54(10):2390
14. Kasahara S, Kawahara J (2016) Effect of bitcoin fee on transaction-confirmation process. [arXiv:1604.00103](https://arxiv.org/abs/1604.00103)
15. Li J, Yuan Y, Wang S, Wang F-Y (2018) Transaction queuing game in bitcoin blockchain. In: IEEE intelligent vehicles symposium (IV). IEEE, pp 114–119
16. Transaction data [EB/OL]. <https://www.cnblogs.com/zhaoweimei/p/transaction.html>

17. Bitcoin Core. Bitcoin core 22.0 release notes [EB/OL]. <https://github.com/bitcoin/bitcoin/blob/master/doc/release-notes/release-notes-22.0.md>
18. Erhardt M (2020) An evaluation of coin selection strategies. Master thesis
19. Diroff DJ (2019) Bitcoin coin selection with leverage [Online]. [arXiv:1911.01330](https://arxiv.org/abs/1911.01330) [cs.DS]
20. Abramova S, Böhme R (2020) Your money or your privacy: a systematic approach to coin selection. *Cryptoeconomic Systems*, vol 20 [Online]. <https://assets.pubpub.org/dwq1f9g6/71581340076503.pdf>
21. Poli R, Kennedy J, Blackwell T (2007) Particle swarm optimization. *Swarm Intell* 1(1):33–57
22. Li K, Zhang T, Wang R (2020) Deep reinforcement learning for multiobjective optimization. *IEEE Trans Cybern* 51(6):3103–3114
23. Tian Z, Jia-Xiang LI (2013) Approach for course of actions determination in influence nets based on greedy algorithm. *Command Control & Simulation*
24. Yan-qin M (2013) Greedy algorithm-based improved adaptive genetic algorithm and its application. *Value Eng* 2013:22
25. Guo P, Wang X, Han Y (2010) The enhanced genetic algorithms for the optimization design. In: 2010 3rd international conference on biomedical engineering and informatics, vol 7. IEEE, pp 2990–2994
26. Deng X (2008) Application of adaptive genetic algorithm in inversion analysis of permeability coefficients. In: 2008 second international conference on genetic and evolutionary computing. IEEE, pp 61–65
27. Nguyen V-H, Trang H-S, Nguyen Q-T, Huynh-Tuong N, Le T-V (2018) Building mathematical models applied to utxos selection for objective transactions. In: 2018 5th NAFOSTED conference on information and computer science (NICS). IEEE, pp 160–164

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.